Supplementary Information: Physics World Models for Computational Imaging

Chengshuai Yang

Contents

1 Supplementary Note 1: Triad Law Mathematical Derivations

The Triad Law formalizes three successive gates that every computational-imaging reconstruction must pass. We derive rigorous bounds for each gate and show how the *recovery ratio* ρ summarizes the overall reconstruction quality.

1.1 Gate 1 — Information-Theoretic Limit (Compression)

Let the forward operator $\mathbf{H} \in \mathbb{R}^{m\times n}$ map an n-dimensional scene $\mathbf{x}$ to m measurements $\mathbf{y}$. Define the *compression ratio* $\gamma = m/n$.

[Compression bound] For any estimator $\hat{\mathbf{x}}(\mathbf{y})$, the minimum achievable mean-squared error satisfies

$$\mathrm{MSE}_{\min} \;\geq\; \frac{1}{n}\sum_{i=1}^{n-m} \sigma_i^2(\mathbf{x}), \tag{S1}$$

where $\sigma_i^2(\mathbf{x})$ denotes the variance of $\mathbf{x}$ along the i-th null-space direction of $\mathbf{H}$.

Interpretation. The null space of $\mathbf{H}$ has dimension $\dim\mathcal{N}(\mathbf{H}) = n - \mathrm{rank}(\mathbf{H}) \geq n - m$. Information lost in the null space cannot be recovered without a prior; Gate 1 therefore sets an *information-theoretic floor* on PSNR:

$$\mathrm{PSNR}_{\max}^{(G1)} = 10\log_{10}\!\left(\frac{\|\mathbf{x}\|_\infty^2}{\mathrm{MSE}_{\min}}\right). \tag{S2}$$

Modalities with higher compression ratios (e.g. CACTI with B frames compressed into one) face a proportionally lower ceiling.

1.2 Gate 2 — SNR-Dependent PSNR Bound (Noise)

Given additive noise $\mathbf{n} \sim \mathcal{N}(0, \sigma_n^2\mathbf{I})$, the measurement SNR is $\mathrm{SNR} = \|\mathbf{H}\mathbf{x}\|^2/(m\,\sigma_n^2)$.

[Noise bound] Under matched forward model and Gaussian noise, the best achievable PSNR is

$$\mathrm{PSNR}_{\max}^{(G2)} = 10\log_{10}(\mathrm{SNR}) + C_{\mathcal{M}}, \tag{S3}$$

where $C_{\mathcal{M}}$ is a modality-dependent constant that absorbs the conditioning of $\mathbf{H}$ and the prior strength:

$$C_{\mathcal{M}} = 10\log_{10}\!\left(\frac{n\,\|\mathbf{x}\|_\infty^2}{\|\mathbf{x}\|^2}\cdot\kappa^{-2}(\mathbf{H})\right), \tag{S4}$$

with $\kappa(\mathbf{H})$ the condition number of $\mathbf{H}$ restricted to its column space.

Interpretation. Gate 2 is a *noise ceiling*: no algorithm can exceed $\mathrm{PSNR}_{\max}^{(G2)}$ regardless of computational budget. Empirically, modalities such as MRI ($C_{\mathrm{MRI}} \approx 8\,\mathrm{dB}$) are more noise-tolerant than CASSI ($C_{\mathrm{CASSI}} \approx 3\,\mathrm{dB}$) due to favorable operator conditioning.

1.3 Gate 3 — Sensitivity to Model Mismatch (Calibration)

Let $\boldsymbol{\theta} \in \mathbb{R}^p$ collect the calibration parameters (e.g. mask shift, PSF blur, coil sensitivity) and let $\mathbf{H}_{\boldsymbol{\theta}}$ denote the parameterized forward operator.

[Calibration sensitivity] For small perturbation $\delta\boldsymbol{\theta}$ around the true parameter $\boldsymbol{\theta}^*$, the PSNR degradation is

$$\Delta\mathrm{PSNR} \approx -\frac{10}{\ln 10}\,\frac{\delta\boldsymbol{\theta}^\top\mathbf{J}^\top\mathbf{J}\,\delta\boldsymbol{\theta}}{\mathrm{MSE}_0}, \tag{S5}$$

where $\mathbf{J} = \partial(\mathbf{H}_{\boldsymbol{\theta}}\mathbf{x})/\partial\boldsymbol{\theta}\big|_{\boldsymbol{\theta}^*}$ is the parameter Jacobian and MSE_0 is the noise-only MSE at $\boldsymbol{\theta}^*$.
Per-parameter sensitivity. Restricting to a single parameter θ_j:

$$\frac{d\,\mathrm{PSNR}}{d\,\theta_j} = -\frac{10}{\ln 10}\,\frac{\|\mathbf{J}_{:,j}\|^2}{\mathrm{MSE}_0}\,\delta\theta_j + \mathcal{O}(\delta\theta_j^2). \tag{S6}$$

The first-order Taylor expansion shows that PSNR degrades linearly in $|\delta\theta_j|$ for small mismatches and quadratically for larger ones.

1.4 Recovery Ratio

We define the *recovery ratio* as

$$\rho = \frac{\mathrm{PSNR}_{\mathrm{achieved}} - \mathrm{PSNR}_{\mathrm{mismatch}}}{\mathrm{PSNR}_{\mathrm{ideal}} - \mathrm{PSNR}_{\mathrm{mismatch}}}, \qquad 0 \le \rho \le 1. \tag{S7}$$

In rare cases where Scenario III exceeds Scenario I (e.g., due to regularization benefits from the corrected operator), ρ may exceed 1.

Under convex reconstruction loss with matched regularization and convex constraint on $\boldsymbol{\theta}$, the recovery ratio satisfies $0 \le \rho \le 1$ for any estimator.

[Proof sketch] The oracle PSNR is the global optimum of the jointly convex problem in $(\mathbf{x}, \boldsymbol{\theta})$. The mismatch PSNR uses a fixed $\boldsymbol{\theta}_0 \neq \boldsymbol{\theta}^*$, yielding a suboptimal feasible point. Any correction step moves towards the optimum, so $\mathrm{PSNR}_{\mathrm{mismatch}} \le \mathrm{PSNR}_{\mathrm{achieved}} \le \mathrm{PSNR}_{\mathrm{oracle}}$, giving $\rho \in [0, 1]$. Under the conditions of Proposition 1, $0 \le \rho \le 1$. In practice, beneficial regularization bias from the corrected operator can yield $\rho > 1$, as observed for CACTI.

2 Supplementary Note 2: Complete OperatorGraph Specification

The `OperatorGraph` is the central abstraction of the PWM framework. It represents a computational imaging pipeline as a directed acyclic graph (DAG) of differentiable operators.

2.1 Node Interface

Every node v in the graph implements the following interface:

Method	Contract
`forward(x)` $\rightarrow$ `y`	Apply the physical forward model. Input shape: `(B, C, *spatial)`. Output shape determined by the operator (e.g. compression changes spatial dims).
`adjoint(y)` $\rightarrow$ `x`	Apply the adjoint (transpose) operator. Must satisfy $\langle \mathbf{Hx}, \mathbf{y} \rangle = \langle \mathbf{x}, \mathbf{H}^\top \mathbf{y} \rangle$ to numerical precision ($< 10^{-6}$ relative error).
`shape_in` / `shape_out`	Static shape annotations for compile-time validation.
`dtype`	Data type contract: `float32` (default) or `complex64`.
`parameters()` $\rightarrow$ `dict`	Returns learnable calibration parameters with their current values.

2.2 Edge Semantics

An edge $e = (u, v)$ represents data flow: the output tensor of node u is fed as input to node v. Edges carry the following metadata:

- **Tensor shape**: inferred at compile time via shape propagation.
- **Data type**: must match between source output and target input.
- **Optional transform**: lightweight reshape or permute operations (e.g. channel stacking for multi-frame CACTI data).

2.3 Compilation Algorithm

Given an `OperatorGraph` $G = (V, E)$:

1. **Topological sort.** Compute a valid execution order $v_1, v_2, \ldots, v_{|V|}$ using Kahn's algorithm. Raise `CycleError` if the graph contains a cycle.
2. **Shape propagation.** Starting from the source node(s), propagate tensor shapes through each operator's `shape_out` to validate all edge contracts.
3. **Automatic adjoint chain.** Construct the adjoint graph $G^\top$ by reversing all edges and replacing each node's `forward` with `adjoint`. The adjoint graph is used for gradient-based reconstruction and the Triad Law sensitivity analysis.

4. **Fusion (optional).** Consecutive linear operators are fused into a single matrix–vector product when dimensions permit, reducing memory traffic.

2.4 JSON Serialization Schema

The graph is serialized to a JSON document for reproducibility and sharing:

```
{
  "version": "0.3.0",
  "nodes": [
    {
      "id": "mask",
      "type": "CassiMask",
      "params": {"shift_r": 0.0, "shift_c": 0.0},
      "shape_in": [1, 28, 256, 256],
      "shape_out": [1, 1, 256, 310]
    }
  ],
  "edges": [
    {"src": "mask", "dst": "detector", "dtype": "float32"}
  ],
  "metadata": {
    "modality": "CASSI",
    "created": "2025-12-01T00:00:00Z"
  }
}
```

3 Supplementary Note 3: All 16 Correction Configurations

Table S1 reports the correction results for all 16 correction configurations across the validated modalities. The first 9 modalities have full end-to-end validation; the remaining 7 are designated Phase 2/4 additions with partial or planned validation.

Table S1: Complete correction parameter summary. ΔPSNR is the improvement from Scenario II (mismatch) to Scenario III (corrected). CASSI correction results in this table use GAP-TV as the reconstruction solver; the main-text deep dive reports results for GAP-TV, MST-L, and HDNet. Scenario I (ideal) PSNR values and recovery ratios (ρ) for all configurations are available in the RunBundle manifests in the code repository.

Modality	**Mismatch Parameter**	**Sc. II (dB)**	**Sc. III (dB)**	**ΔPSNR (dB)**	**RMSE**
Matrix[1]	gain_bias	11.14	23.35	+12.21	0.0042
CT	center_of_rotation	13.41	24.09	+10.68	0.0031
CACTI	mask_timing	14.48	37.42	+22.94	0.0018
Lensless	psf_shift	23.48	27.03	+3.55	0.0089
MRI	coil_sensitivities	6.94	55.19	+48.25	0.0003
SPC	gain_bias	11.14	23.35	+12.21	0.0042
CASSI (Alg 1)[2]	mask_geo + dispersion	20.96	21.50	+0.54	0.0105
CASSI (Alg 2)[3]	mask_geo + dispersion	20.96	21.72	+0.76	0.0098
Ptychography	position_offset	17.35	24.44	+7.09	0.0063
Phase 2/4 additions — validation in progress					
Holography	propagation_distance	—	—	—	—
FPM	LED_positions	—	—	—	—
Phase Retrieval	support_constraint	—	—	—	—
OCT	dispersion_coeffs	—	—	—	—
Fluorescence	PSF_aberration	—	—	—	—
Ultrasound	speed_of_sound	—	—	—	—
Radar/SAR	motion_compensation	—	—	—	—

[1]Matrix (Generic) is a canonical test configuration that uses the same gain-bias operator template as SPC; identical results confirm pipeline consistency rather than representing an independent modality.

[2]Reported values are averaged over 10 test scenes using GAP-TV as the reconstruction solver. Sc. II from InverseNet validation; Sc. III values are oracle upper bounds (true operator applied to mismatched data).

[3]Note: the correction-table PSNR reflects GAP-TV reconstruction; state-of-the-art solvers (e.g., MST-L) achieve 27.33 dB oracle PSNR under the corrected operator (see main text).

4 Supplementary Table S2: CASSI Per-Scene Results

Table S2 reports PSNR (dB) for 10 KAIST scenes across five representative methods under four scenarios.[4]

Table S2: CASSI per-scene PSNR (dB). Sc. I = ideal forward model; Sc. II = mismatched (baseline); Sc. III-A1 = correction Algorithm 1; Sc. IV (Oracle) = ground-truth operator through full pipeline. GAP-TV values for Sc. I, Sc. II, and Sc. IV are from InverseNet validation[1]. Sc. III-A1 entries marked * use InverseNet Oracle Mask values (true operator on mismatched data) as upper bounds; Algorithm 1 re-validation under the 5-parameter mismatch is pending. Sc. IV reproduces Sc. I by design (±0.01 dB). Other methods (TwIST, DeSCI, MST, HDNet) have *not* been re-validated under the InverseNet mismatch and retain values from prior experiments with a different mismatch configuration. Algorithm 2 results are reported in the correction parameter table (Table S1).

Scene	Method	Sc. I	Sc. II	Sc. III-A1	Sc. IV
scene01	GAP-TV	26.49	24.08	24.18*	26.49
scene01	TwIST	27.18	15.79	21.34	27.18
scene01	DeSCI	28.01	14.92	20.15	28.01
scene01	MST	33.47	16.84	22.61	33.47
scene01	HDNet	34.12	17.02	23.10	34.12
scene02	GAP-TV	24.60	21.89	22.82*	24.60
scene02	TwIST	25.73	15.21	20.48	25.73
scene02	DeSCI	26.95	14.38	19.54	26.95
scene02	MST	31.84	16.12	21.73	31.84
scene02	HDNet	32.56	16.45	22.01	32.56
scene03	GAP-TV	25.96	18.62	19.68*	25.96
scene03	TwIST	28.04	16.35	21.87	28.04
scene03	DeSCI	29.22	15.64	20.93	29.22
scene03	MST	34.67	17.48	23.25	34.67
scene03	HDNet	35.31	17.71	23.68	35.31
scene04	GAP-TV	28.36	23.37	24.41*	28.36
scene04	TwIST	31.09	18.02	23.75	31.09
scene04	DeSCI	32.14	17.11	22.64	32.14
scene04	MST	37.82	19.33	25.47	37.82
scene04	HDNet	38.45	19.68	25.94	38.45
scene05	GAP-TV	23.66	20.39	21.27*	23.66
scene05	TwIST	26.54	15.53	20.84	26.54
scene05	DeSCI	27.68	14.72	19.87	27.68
scene05	MST	32.91	16.67	22.14	32.91
scene05	HDNet	33.58	16.94	22.53	33.58
scene06	GAP-TV	22.34	20.39	21.00*	22.34
scene06	TwIST	29.18	16.74	22.31	29.18
scene06	DeSCI	30.42	15.97	21.24	30.42
scene06	MST	35.89	18.05	23.87	35.89
scene06	HDNet	36.54	18.32	24.25	36.54

[4]The solver subset used in this supplementary per-scene table (GAP-TV, TwIST, DeSCI, MST, HDNet) differs from the three methods reported in the main-text CASSI deep dive (GAP-TV, MST-L, HDNet). The supplementary tables use an earlier solver subset selected during initial experiments; GAP-TV values have been updated from InverseNet validation. The supplementary table uses MST (base variant); the main text reports MST-L (large variant). Different model sizes account for the PSNR difference.

Scene	**Method**	**Sc. I**	**Sc. II**	**Sc. III-A1**	**Sc. IV**
scene07	GAP-TV	23.51	20.56	21.07*	23.51
scene07	TwIST	24.76	14.32	19.64	24.76
scene07	DeSCI	25.84	13.55	18.73	25.84
scene07	MST	30.72	15.48	21.02	30.72
scene07	HDNet	31.35	15.73	21.38	31.35
scene08	GAP-TV	22.16	20.57	21.10*	22.16
scene08	TwIST	28.53	16.52	22.04	28.53
scene08	DeSCI	29.71	15.78	21.12	29.71
scene08	MST	35.14	17.83	23.55	35.14
scene08	HDNet	35.79	18.08	23.94	35.79
scene09	GAP-TV	23.03	19.14	20.61*	23.03
scene09	TwIST	26.89	15.64	21.02	26.89
scene09	DeSCI	28.04	14.91	20.08	28.04
scene09	MST	33.28	16.87	22.35	33.28
scene09	HDNet	33.94	17.12	22.71	33.94
scene10	GAP-TV	23.27	20.58	21.04*	23.27
scene10	TwIST	30.34	17.54	23.21	30.34
scene10	DeSCI	31.52	16.73	22.14	31.52
scene10	MST	37.05	18.89	24.91	37.05
scene10	HDNet	37.68	19.15	25.34	37.68

5 Supplementary Table S3: Full 26-Modality Benchmark

Table S3 lists all 26 modalities in the PWM benchmark, grouped by physical carrier tier.

Table S3: Full 26-modality benchmark. PSNR values are representative (Scenario I, default solver). "Ref. PSNR" is the best published result where available.

#	Modality	Domain	PSNR (dB)	Ref. PSNR (dB)	Status	Solver
1	Matrix (toy)	Incoherent	23.35	—	Validated	ADMM-TV
2	SPC	Incoherent	23.35	24.1	Validated	ADMM-TV
3	CASSI	Incoherent	24.34	34.2	Validated	GAP-TV / MST-L
4	CACTI	Incoherent	35.33	33.4	Validated	GAP-TV
5	Lensless	Incoherent	27.03	28.5	Validated	ADMM-TV
6	Fluorescence	Incoherent	—	30.1	Phase 4	—
7	Holography	Coherent	—	32.7	Phase 2	—
8	Ptychography	Coherent	24.44	26.8	Validated	ePIE
9	FPM	Coherent	—	31.5	Phase 2	—
10	Phase Retrieval	Coherent	—	29.3	Phase 2	—
11	CDI	Coherent	—	27.4	Phase 4	—
12	OCT	Coherent	—	35.2	Phase 4	—
13	CT (parallel)	X-ray	24.09	42.1	Validated	FBP + TV
14	CT (fan-beam)	X-ray	—	40.8	Phase 2	—
15	CT (cone-beam)	X-ray	—	39.5	Phase 2	—
16	Electron Pty.	Electron	—	25.9	Phase 4	—
17	MRI (Cartesian)	RF	55.19	42.3	Validated	CG-SENSE
18	MRI (radial)	RF	—	38.7	Phase 2	—
19	MRI (spiral)	RF	—	37.2	Phase 4	—
20	Ultrasound	Acoustic	—	31.8	Phase 4	—
21	Radar / SAR	RF	—	28.4	Phase 4	—
22	NeRF	Multi-view	—	31.7	Phase 2	—
23	3DGS	Multi-view	—	33.2	Phase 2	—
24	Light Field	Multi-view	—	36.1	Phase 4	—
25	Structured Illum.	Incoherent	—	32.4	Phase 4	—
26	Ghost Imaging	Incoherent	—	22.8	Phase 4	—

6 Supplementary Table S4: YAML Registry Summary

Table S4 summarizes the nine YAML registry files (totalling 7,034 lines) that define the PWM framework's modular architecture.

Table S4: YAML registry files in `packages/pwm_core/contrib/`.

#	File	Lines	Purpose
1	`modalities.yaml`	1200	Defines 64 modality entries with keywords, forward model equations, and default solvers.
2	`graph_templates.yaml`	1400	OperatorGraph skeletons for 89 templates across 64 modalities.
3	`photon_db.yaml`	624	Photon models parameterized by source power, quantum efficiency, exposure, and detector.
4	`mismatch_db.yaml`	797	Mismatch parameters, perturbation ranges, and correction methods per modality.
5	`compression_db.yaml`	1186	Recoverability tables mapping compression ratio to expected PSNR with provenance.
6	`solver_registry.yaml`	650	Maps solver names to implementations (ADMM^2, FISTA^3, GAP-TV, PnP, learned).
7	`primitives.yaml`	450	Primitive operator metadata (type, adjoint availability, differentiability).
8	`dataset_registry.yaml`	380	Links modalities to benchmark datasets (KAIST, fastMRI, etc.).
9	`acceptance_thresholds.yaml`	347	Pass/fail thresholds per metric for automated validation.

7 Supplementary Note 4: RunBundle Schema

The `RunBundle` is PWM's reproducibility artifact. Every experiment produces a RunBundle that captures the complete computational state.

7.1 RunBundle v0.3.0 Specification

Table S5: RunBundle v0.3.0 field specification.

Field	Type	Description
`version`	string	Schema version, currently `"0.3.0"`.
`git_hash`	string	Full 40-character SHA-1 of the commit used to generate results.
`rng_seeds`	object	Random seeds for Python (`random`, `numpy`), PyTorch (CPU, CUDA), and any solver-specific RNG.
`platform_info`	object	OS, Python version, GPU model, CUDA version, PyTorch version, and `pip freeze` hash.
`sha256_hashes`	object	SHA-256 checksums of all input data files (measurements, masks, ground truth).
`metrics`	object	Computed quality metrics: PSNR, SSIM, LPIPS, and any modality-specific metrics.
`timestamps`	object	ISO-8601 timestamps for start, end, and per-stage completion.
`operator_state`	object	Serialized OperatorGraph including all node parameters and edge metadata.
`triad_report`	object	Triad Law evaluation: gate pass/fail status, PSNR bounds, and recovery ratio ρ.
`correction_trajectory`	array	Time series of correction iterations: [{`iter`, `params`, `loss`, `psnr`, `grad_norm`}, ...]. Enables convergence analysis and early-stopping diagnostics.
`solver_config`	object	Complete solver hyperparameters (algorithm, iterations, step size, regularization weight, denoiser checkpoint).
`modality`	string	Modality identifier matching `modalities.yaml`.
`scenario`	string	One of `I` (ideal), `II` (mismatch), `III` (corrected), `IV` (oracle).
`notes`	string	Free-form text field for experiment annotations.

7.2 Integrity Verification

A RunBundle can be verified by:

1. Checking `git_hash` matches the repository state.
2. Re-computing `sha256_hashes` from the data files.
3. Re-running the solver with the stored `rng_seeds` and `solver_config` and comparing metrics within tolerance (± 0.01 dB PSNR).

8 Supplementary Note 5: Computational Cost Analysis

8.1 Runtime per Modality

All timings were measured on a single NVIDIA RTX 4090 GPU (24 GB VRAM) with PyTorch 2.1 and CUDA 12.1.

Table S6: Computational cost for correction (Scenario II → III). RoIC = dB recovered per GPU-hour.

Modality	**Iterations**	**Runtime (s)**	**ΔPSNR (dB)**	**RoIC (dB/GPU-hr)**
Matrix	200	12	+12.21	3663.0
CT	500	85	+10.68	452.0
CACTI	1000	180	+22.94	458.8
Lensless	300	45	+3.55	284.0
MRI	150	30	+48.25	5790.0
SPC	200	15	+12.21	2929.6
CASSI (Alg 1)	500	300	+0.54	6.5
CASSI (Alg 2)	2000	3200	+0.76	0.9
Ptychography	800	420	+7.09	60.8

8.2 RoIC Metric: Efficiency of Correction

We define the *Return on Invested Compute* (RoIC) as:

$$\text{RoIC} = \frac{\Delta\text{PSNR (dB)}}{T_{\text{GPU}}\ \text{(hours)}}, \tag{S8}$$

where T_{GPU} is the wall-clock GPU time.

Key observations:

- **MRI** achieves the highest RoIC (5790 dB/GPU-hr) because coil sensitivity correction yields massive PSNR gains with a well-conditioned linear operator that converges rapidly.
- **Matrix/SPC** also show high RoIC due to the simplicity of the gain–bias correction model.
- **CASSI Algorithm 2** has the lowest RoIC (0.9 dB/GPU-hr) because it jointly optimizes mask geometry and dispersion over 2000 iterations, each requiring a full forward–adjoint cycle through the dispersive operator. Under the multi-parameter InverseNet mismatch, the absolute correction gains are modest (+0.76 dB), reflecting the inherent difficulty of simultaneously correcting five coupled mismatch parameters.
- The practical implication is that Algorithm 1 (fixed dispersion, correct mask geometry only) is preferred when compute budget is limited, recovering 71% of Algorithm 2's ΔPSNR at <10% of the cost.

8.3 Scaling Considerations

For large-scale deployment:

- Correction is *embarrassingly parallel* across scenes/slices — multi-GPU scaling is near-linear.

- The OperatorGraph compilation overhead is amortized: once compiled, the same graph serves all scenes of a given modality.
- Memory footprint scales with the measurement dimension m, not the scene dimension n, making correction feasible even for high-resolution 3D volumes (e.g. 512^3 CT).

References

[1] Yang, C. InverseNet: Benchmarking operator mismatch in snapshot compressive imaging. Technical Report, NextGen PlatformAI C Corp (2026).

[2] Boyd, S., Parikh, N., Chu, E., Peleato, B. & Eckstein, J. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning* **3**, 1–122 (2011).

[3] Beck, A. & Teboulle, M. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences* **2**, 183–202 (2009).